

Master Thesis

Design and Evaluation of Efficient Semantic Company Retrieval at Scale

Marmee Pandya
(matriculation number 1963521)

July 7, 2026

Submitted to
Chair of Information Systems
Dr. Ralph Peeters
University of Mannheim

Abstract

Your thesis must contain an abstract. A good reference for thesis writing is [Zobel \(2014\)](#); we highly recommend that you study this or a similar book during your studies. He writes the following about the abstract:

An abstract is typically a single paragraph of about 50 to 200 words. The function of an abstract is to allow readers to judge whether or not the paper is of relevance to them. It should therefore be a concise summary of the paper's aims, scope, and conclusions. There is no space for unnecessary text; an abstract should be kept to as few words as possible while remaining clear and informative. Irrelevancies, such as minor details or a description of the structure of the paper, are inappropriate, as are acronyms, abbreviations, and mathematics. Sentences such as "We review relevant literature" should be omitted.

The more specific an abstract is, the more interesting it is likely to be. Instead of writing "space requirements can be significantly reduced", write "space requirements can be reduced by 60%". Instead of writing "we have a new inversion algorithm", write "we have a new inversion algorithm, based on moveto-front lists".

Many scientists browse research papers outside their area of expertise. You should not assume that all likely readers will be specialists in the topic of their paper-abstracts should be self-contained and written for as broad a readership as possible. Only in rare circumstances should an abstract cite another paper (for example, when one paper consists entirely of analysis of results in another), in which case the reference should be given in full, not as a citation to the bibliography. ([Zobel 2004](#))

Contents

Abstract	ii
1. Project Overview	1
2. ISTARI.AI and its Data	2
2.1. About ISTARI.AI	2
2.2. The Global Organisation Index (GOI)	2
2.3. Data Pipeline and Methodology	2
2.3.1. Schema and Key Fields	3
2.3.2. Industry Classification: NACE Codes	4
2.3.3. Organisation Size Classification	4
2.3.4. Geographic Coverage	4
2.3.5. Evaluation Dataset	5
3. Literature	6
4. Experiments	7
4.1. Experimental Setup	7
4.1.1. Dataset	7
4.1.2. Evaluation Protocol	7
4.1.3. Computational Environment	8
4.2. Sparse Retrieval Baseline: BM25	8
4.2.1. Implementation	8
4.2.2. Qualitative Analysis	9
4.3. Dense Retrieval Baselines	9
4.3.1. all-MiniLM-L6-v2 (MiniLM)	9
4.3.2. BGE-large-en-v1.5 (BGE)	9
4.3.3. OpenAI text-embedding-3-small (OpenAI)	10
4.4. Results	10
4.4.1. Retrieval Quality	10
4.5. Runtime Analysis	12
4.6. Evaluation Protocol Change for Follow-Up Experiments	13
4.7. Extended Baseline Comparison: Additional Embedding Models	14
4.8. BGE Instruction-Prefix Correction	15
4.9. Query Expansion: HyDE Pipeline	16
4.10. Hybrid Retrieval: Reciprocal Rank Fusion	16
4.11. LLM-as-Judge Relevance Validation	17
4.12. Production API Evaluation and MiniLM Rerank	18

Contents

4.13. ANN Index Scaling to 2 Million Vectors	18
4.14. Recall@1000 Error Analysis	19
4.15. Oracle Union Recall Ceiling	20
5. Introduction	21
6. Literature Review	23
6.1. Preliminaries	24
6.2. Related Work on A	24
6.3. Related Work on B	24
6.4. Related Work on C	24
6.5. Summary	24
7. Body Chapter 1	25
8. Body Chapter 2	27
9. Experimental Evaluation	29
9.1. Experimental Setup	29
9.2. Results 1	29
9.3. Results 2	29
9.4. Results 3	29
9.5. Discussion	29
10. Conclusions	30
Bibliography	31
A. Additional Experimental Results	33
B. Proof Details	34
Ehrenwörtliche Erklärung	35

1. Project Overview

Thesis Goal: Build and evaluate an efficient embedding-based semantic retrieval system for large-scale company data that outperforms the current production search system.

Core Question: Can embedding-based semantic retrieval combined with approximate nearest neighbor search efficiently generate accurate top-k candidate company sets from large-scale datasets while maintaining low query latency?

Current Todo List: - Literature Review

IMPORTANT::: MUST READ AT THE TIME TO WRITING THE INTRODUCTION. Aa current introduction page bani sake che. Ema last je bhi paragraph hase ema badha j sections/chapters explain krwana ke chapter 2 ma aa and chapter 3 ma aa and so on. Ek paragraph eno j banse.

2. ISTARI.AI and its Data

2.1. About ISTARI.AI

This thesis is conducted in collaboration with ISTARI.AI¹, a data intelligence company that specialises in providing high-quality firmographic data at scale. Istari’s core product is a curated, organisation-level dataset that covers organisations worldwide, enabling use cases such as market analysis, company search, and lead generation. The central research challenge addressed in this thesis is improving the quality of semantic company retrieval, arises directly from Istari’s operational needs.

2.2. The Global Organisation Index (GOI)

The primary dataset used in this thesis is Istari’s **Global Organisation Index** (GOI), a curated collection of approximately 20 million active organisations spanning 232 countries and territories. The GOI serves as the canonical source of firmographic data within Istari’s data infrastructure, covering organisation identity, location, industry classification, size, etc.

2.3. Data Pipeline and Methodology

The GOI is constructed through a multi-stage validation pipeline that deliberately prioritises data quality over volume. The pipeline proceeds as follows:

1. **Registry Collection.** National registers are systematically queried worldwide, including commercial registers, company registers, and association registers. These primary sources are enriched with additional open data sources, yielding an initial pool of approximately 400 million organisations.
2. **Domain Attribution.** Each organisation is assessed for attribution to a clearly identifiable web domain. Approximately 10% of the initial pool, around 40 million organisations can be matched to a domain. The remainder are excluded as they are either no longer active, were never truly operational (e.g., pure holding structures), or lack a web presence.
3. **Activity Verification.** The 40 million domain-attributed organisations are further filtered by verifying that their domains are still actively operated. This final

¹https://docs.istari.ai/goi/goi_start_page/

2. ISTAR.AI and its Data

step reduces the dataset to approximately 20 million organisations that are demonstrably active.

This pipeline ensures that every record in the GOI is both web domain-verified and confirmed active, a key differentiator from traditional firmographic databases, which typically retain large numbers of inactive or dormant records.

2.3.1. Schema and Key Fields

Each record in the GOI contains a set of core identity fields, geographic fields, and classification fields. The fields most relevant to this thesis are summarised in Table 2.1.

Table 2.1.: Key fields of the GOI schema used in this thesis.

Field	Type	Description
name	STRING	Organisation name
domain	STRING	Organisation website domain (unique identifier)
summary	STRING	AI-generated textual summary of the organisation
keywords	LIST	Keywords
employee_class	STRING	Employee count bracket
country	STRING	Country
country_code	STRING	ISO Country Code
state / state_code	STRING	State or province
region / region_code	STRING	Region
district / district_code	STRING	District
municipality / municipality_code	STRING	Municipality
address	STRING	Full address
latitude	STRING	Latitude coordinate
longitude	STRING	Longitude coordinate
nace_code	STRING	NACE industry classification code
organization_type	STRING	Company, Startup, Academic, Public, or Other
organization_size	STRING	Micro, Small, Medium-sized, or Large enterprise
country	STRING	Country of registration

Of particular importance for this thesis is the **summary** field. This field contains an AI-generated natural language description of each organisation, synthesised from its web presence and available structured data. As explored in the data analysis phase of this work **TODO: fix the section**(see Section ??), the **summary** field is the most semantically rich and consistently populated field in the dataset, with effectively zero missing values across the corpus used for experiments. It therefore serves as the primary input for all retrieval models evaluated in this thesis.

2.3.2. Industry Classification: NACE Codes

Industries in the GOI are classified using the **NACE** taxonomy² (*Nomenclature statistique des activités économiques dans la Communauté européenne*), the European Union’s standard statistical classification of economic activities. The GOI covers 22 NACE sectors. Table 2.2 lists the ten most represented sectors in the dataset.

Table 2.2.: Top 10 industries in the GOI by organisation count.

NACE	Industry	Count	Share (%)
N	Professional, scientific & technical activities	3,093,879	18.03
G	Wholesale & retail trade	2,240,191	13.05
C	Manufacturing	2,006,435	11.69
R	Human health & social work	1,440,340	8.39
K	Telecom, IT & computing	1,269,131	7.39
F	Construction	1,173,226	6.84
I	Accommodation & food service	1,126,594	6.56
S	Arts, sports & recreation	1,101,592	6.42
Q	Education	584,430	3.41
J	Publishing, broadcasting & content	554,985	3.23

2.3.3. Organisation Size Classification

Organisation size in the GOI is derived from employee and revenue signals and bucketed into four categories following EU SME definitions, as shown in Table 2.3.

Table 2.3.: Organisation size distribution in the GOI.

Size	Employee Range	Count	Share (%)
Micro	0–9	7,341,838	42.78
Small	10–49	7,494,409	43.67
Medium-sized	50–249	1,501,655	8.75
Large enterprise	250+	824,126	4.80

The distribution is heavily skewed towards smaller organisations: over 86% of the GOI consists of micro and small enterprises. This has implications for retrieval, as queries targeting large enterprises represent a smaller and more distinctive subset of the corpus.

2.3.4. Geographic Coverage

The GOI covers 232 countries and territories. Table 2.4 lists the top five countries by volume.

²<https://ec.europa.eu/eurostat/web/nace>

2. ISTARI.AI and its Data

Table 2.4.: Top 5 countries by organisation count in the GOI.

Country	Organisations
United States	3,626,794
Germany	1,828,181
United Kingdom	1,013,026
Netherlands	778,416
Italy	625,754

It is worth noting that approximately 2.7 million organisations in the GOI do not have a standardised administrative region. These records are intentionally retained as they remain valuable for non-geographic analyses such as industry or domain-level retrieval, the primary focus of this thesis.

2.3.5. Evaluation Dataset

TODO: Delete this topic?

For the experiments conducted in this thesis, the full 20 million company corpus is not directly used for retrieval in the baseline phase. Instead, Istari provided an evaluation dataset consisting of 101 real-world search queries along with their corresponding production search results (top 1,000 ranked companies per query). This yielded 101,000 (query, company) pairs in total.

Where the same domain appears linked to multiple addresses, the GOI deduplicates by domain and retains the record with the highest employee count, treating it as the organisation’s headquarters. This policy is relevant for the experimental corpus used in this thesis. After deduplication by domain, this evaluation dataset covers 98,716 unique companies - a representative subset of the GOI. All baseline retrieval experiments (BM25, sentence-embedding, and hybrid retrieval) are conducted over this corpus, with the production top-100 results per query serving as pseudo-relevance labels for evaluation. Scaling retrieval to the full 20 million corpus is planned for the later phases of the thesis using GPU-accelerated infrastructure provisioned by the supervisor. **TODO: Need to add section and also 98K is related to this or not.**

3. Literature

4. Experiments

This chapter describes the experimental setup, baseline retrieval systems, evaluation methodology, and results obtained for the company search task over Istari’s Global Organisation Index (GOI). Section 4.1 introduces the dataset and evaluation protocol. Sections 4.2–4.3 describe each retrieval baseline in turn. Section 4.4 presents and analyses the results across all methods, and Section 4.5 provides a runtime comparison.

4.1. Experimental Setup

4.1.1. Dataset

The evaluation corpus is derived from Istari’s production search logs. A set of 101 representative queries was provided by Istari, covering a wide range of company search intents, including industry-specific queries (e.g. “*pharmaceutical companies in the US*”), geographic queries (e.g. “*tech startups in Berlin*”), and functional queries (e.g. “*B2B SaaS companies*”). For each query, the production system returned its top-1 000 ranked results, yielding 101 000 query–company pairs in total. After de-duplication across queries, the corpus consists of **98 716 unique companies**, each represented by a domain name, a company name, and an AI-generated natural-language summary of the company’s business activity.

The summary field was selected as the primary retrieval field after an exploratory data analysis of all available GOI attributes. It was found to be fully populated across the corpus (0% missing values) and to contain the richest semantic content for matching against free-text queries, compared to structured fields such as industry codes or founding year.

4.1.2. Evaluation Protocol

Pseudo-relevance labels. No human-annotated relevance judgements are available for the Istari company search task. Following standard practice in information retrieval research when ground-truth labels are absent (Voorhees 1999), this thesis adopts a *pseudo-relevance* assumption: a company is considered **relevant** for a given query if and only if it appears in the **top-100 results** of Istari’s production search system for that query. This yields a relevance set of at most 100 companies per query, representing $\sim 0.1\%$ of the total corpus.

This assumption has an important implication for the evaluation: the metrics measure the degree to which each retrieval system agrees with the production system, not whether it recovers objectively relevant companies. Consequently, a retrieval system

4. Experiments

that is strictly better than the production system may appear to underperform because it surfaces companies the production system missed. This limitation is acknowledged throughout the analysis.

Metrics. Three standard information retrieval metrics are reported at cut-offs $k \in \{10, 50, 100, 1000\}$:

- **Precision@ k** : the fraction of the top- k retrieved companies that are relevant. This measures user-facing quality — what a user sees on the first page of results.
- **Recall@ k** : the fraction of all relevant companies that appear in the top- k results. This measures coverage — important for downstream re-ranking pipelines that operate on a retrieved candidate set.
- **NDCG@ k** (Normalised Discounted Cumulative Gain): a ranking-aware metric that rewards systems for placing relevant companies higher in the list, using a logarithmic discount. NDCG@ k is the primary quality metric in this thesis, as it directly reflects the experience of a user who interacts primarily with the top results.

All metrics are averaged across all 101 queries. Precision and Recall are always reported at the same cut-off values so that the precision–recall trade-off at each depth is visible.

4.1.3. Computational Environment

All experiments are conducted on the University of Mannheim HPC cluster. Dense model encoding is performed on an NVIDIA A100 GPU with 80 GB HBM2 memory. BM25 retrieval runs on CPU. Query latency measurements are performed on the same hardware and reported as the mean over 20 benchmark queries to obtain stable estimates. Latency is decomposed into three components — local query encoding, FAISS index search, and external network time — so that self-hosted and API-based methods are compared on equal terms.

4.2. Sparse Retrieval Baseline: BM25

4.2.1. Implementation

The BM25 baseline is implemented using the `rank_bm25` Python library (Robertson and Zaragoza 2009), which provides the Okapi BM25 variant with default hyperparameters $k_1 = 1.5$ and $b = 0.75$. All 98 716 company summaries are tokenised using the NLTK punkt tokeniser (Bird et al. 2009) with lower-casing applied, and the resulting token lists are used to construct the BM25 inverted index. At query time, each query is tokenised identically and scored against the index. The top-1 000 companies by BM25 score are retrieved for each query and stored for evaluation. Index construction takes 28.7 seconds and requires no GPU.

4. Experiments

4.2.2. Qualitative Analysis

Manual inspection of BM25 results reveals a consistent failure mode arising from the *vocabulary mismatch problem*: BM25 retrieves companies that contain query keywords in their description, regardless of whether those companies operate in the queried domain.

A representative example is the query “*software companies*”. The top BM25 results are predominantly investment banks and venture capital firms whose summaries frequently mention the word *software* in the context of describing their investment portfolio (e.g. “*we invest in software, fintech, and SaaS businesses*”). These companies are not software companies — they fund them. BM25 cannot distinguish between a company *operating* in a domain and a company *discussing* that domain. This limitation provides a clear and concrete motivation for semantic retrieval methods that operate on the meaning of text rather than on token overlap.

4.3. Dense Retrieval Baselines

Three dense retrieval baselines are evaluated, covering a range of model sizes, computational requirements, and deployment constraints. All three follow the same two-stage pipeline introduced by Karpukhin et al. (2020): (i) encode all company summaries offline into a dense vector index using FAISS (Johnson et al. 2019), and (ii) at query time, encode the query into a vector and retrieve the top-1000 nearest neighbours from the index.

4.3.1. all-MiniLM-L6-v2 (MiniLM)

all-MiniLM-L6-v2 is a lightweight sentence-transformer model (Reimers and Gurevych 2019) with 22 million parameters, producing 384-dimensional embeddings. It is distilled from a larger BERT model and fine-tuned on sentence similarity tasks. All 98 716 company summaries are encoded in a single batch pass, taking 0.6 minutes on the A100 GPU. A `faiss.IndexFlatL2` index (exact Euclidean search) is constructed over the resulting embeddings in 0.2 seconds. MiniLM does not require embedding normalisation.

MiniLM is included as the smallest and most computationally efficient dense baseline, providing a lower bound on dense retrieval quality and establishing whether a minimal dense model is sufficient to outperform BM25.

4.3.2. BGE-large-en-v1.5 (BGE)

BAAI/bge-large-en-v1.5 is a 335-million parameter BERT-large model developed by the Beijing Academy of Artificial Intelligence (Chen et al. 2024), producing 1024-dimensional embeddings. It is specifically optimised for dense retrieval through hard negative mining during fine-tuning — a training strategy in which the model is exposed to query–document pairs that are superficially similar but semantically distinct, forcing it to learn fine-grained semantic distinctions.

4. Experiments

Encoding all 98 716 company summaries takes 7.0 minutes on the A100 GPU at a batch size of 256. BGE requires L2 normalisation of all embeddings (`normalize_embeddings=True`), as it is trained under the cosine similarity objective. Accordingly, a `faiss.IndexFlatIP` index (inner product, equivalent to cosine similarity for unit-normalised vectors) is used rather than `IndexFlatL2`.

At query time, each query is encoded independently and normalised before being passed to the FAISS index. The query encoding step takes 16.6ms on the A100 GPU, and the subsequent FAISS search takes 27.8ms, for a total query latency of 44.4ms.

4.3.3. OpenAI text-embedding-3-small (OpenAI)

`text-embedding-3-small` is a commercial embedding model provided by OpenAI via their REST API. It produces 1536-dimensional embeddings. Unlike the open-source models above, the architecture and training procedure are proprietary. Encoding all 98 716 company summaries is performed via batched API calls (batch size 500) and takes 9.7 minutes, of which the majority is network latency rather than model computation. The resulting embeddings are unit-normalised by the API, and a `faiss.IndexFlatIP` index is constructed locally. The total cost of encoding the full corpus is approximately \$0.15 at the published pricing of \$0.02 per million tokens.

This model is included as the commercial upper-bound baseline, following a suggestion from the industry supervisor that a high-quality API-based embedding model be evaluated alongside open-source alternatives.

Latency note. Query latency for the OpenAI baseline comprises two components: a network round-trip to the OpenAI API servers (211.7ms, measured over 20 queries) and a local FAISS search (42.9ms). The network component is not a property of the model itself — it reflects internet latency between the University of Mannheim cluster and OpenAI’s infrastructure — and should not be interpreted as a measure of model speed. For a fair comparison, the local FAISS search time (42.9ms) is comparable to BGE’s search time (27.8ms), confirming that both methods perform equivalently at the index search stage. The substantive difference is that OpenAI requires an external API call for query encoding, whereas BGE encodes locally on GPU in 16.6ms.

4.4. Results

4.4.1. Retrieval Quality

Table 4.1 presents the full evaluation results for all four methods across all metrics and cut-offs.

BM25 vs dense retrieval. All three dense methods outperform BM25 at low cut-offs. At $k = 10$, BGE and OpenAI achieve an NDCG of 0.227, representing a **37% improvement** over BM25’s 0.166. Precision@10 improves from 0.156 (BM25) to 0.216 (BGE)

4. Experiments

Table 4.1.: Retrieval quality of all four baseline systems averaged over 101 queries. Bold indicates the best value in each column. BGE and OpenAI are statistically indistinguishable across all metrics.

Method	NDCG@ k				Precision@ k				Recall@ k		
	10	50	100	1000	10	50	100	1000	10	50	100
BM25	0.166	0.144	0.134	0.446	0.156	0.136	0.128	0.068	0.016	0.068	0.128
MiniLM	0.192	0.150	0.133	0.432	0.186	0.138	0.122	0.065	0.019	0.069	0.122
BGE	0.227	0.187	0.170	0.527	0.216	0.173	0.159	0.080	0.022	0.087	0.159
OpenAI	0.227	0.174	0.161	0.526	0.224	0.160	0.149	0.080	0.022	0.080	0.149

— a 38% relative gain. This confirms the hypothesis that semantic retrieval better captures the intent of natural-language company search queries, particularly for cases where relevant companies use vocabulary different from the query terms.

At large cut-offs ($k = 1000$), the gap narrows. BM25 achieves a Recall@1000 of 0.682, compared to BGE’s 0.796 and OpenAI’s 0.803. This pattern is consistent with findings in the BEIR benchmark (Thakur et al. 2021): dense models are stronger at ranking precision, while keyword-based methods maintain competitive coverage at large retrieval depths due to their broad lexical matching.

BGE vs OpenAI. BGE and OpenAI are **effectively equivalent** across all metrics and cut-offs. The largest observed difference is 0.013 NDCG points (at $k = 50$), which is within the range of query-level variance and should not be interpreted as a meaningful quality advantage. This finding has an important practical implication: a free, self-hosted model (BGE) achieves the same retrieval quality as a commercial API (OpenAI) on this task.

MiniLM. MiniLM consistently underperforms BGE and OpenAI at all cut-offs, and is only marginally better than BM25 at large k (Recall@1000: 0.653 vs 0.682 for BM25). It is also slower than BGE in production (query latency 80.9 ms vs 44.4 ms), as discussed in Section 4.5. MiniLM is therefore dominated by BGE on both quality and speed, and there is no operating point at which it represents the optimal choice.

Precision and the low-prevalence setting. Absolute precision values are low across all methods (Precision@10 ranging from 0.156 to 0.224). This is expected: relevant companies represent at most 100 out of 98 716 corpus entries — approximately 0.1% of the corpus. In such a low-prevalence setting, even a perfect retrieval system would show low absolute precision at large k . What matters is the relative improvement between methods: BGE and OpenAI achieve 38–43% higher Precision@10 than BM25, which is a meaningful gain.

4. Experiments

The NDCG dip at $k = 100$. An artifact of the evaluation design is visible in Table 4.1: NDCG values decrease from $k = 10$ to $k = 100$ before rising sharply at $k = 1000$ for all methods. This occurs because the pseudo-relevance labels are defined as the production top-100. At exactly $k = 100$, NDCG measures whether all 100 relevant companies are ranked perfectly within the top-100 retrieved results — a maximally strict condition. At $k = 1000$, there is substantially more room to accommodate the relevant set, and the metric rises accordingly. This artifact would not appear with human-annotated relevance labels that include varying degrees of relevance beyond a hard top-100 cut-off.

4.5. Runtime Analysis

Table 4.2 presents the decomposed query latency for all four methods. Latency is broken into three components to enable a fair comparison between self-hosted models and the API-based OpenAI baseline.

Table 4.2.: Query latency decomposed into local encoding, FAISS search, and external network time, averaged over 20 benchmark queries on an NVIDIA A100 GPU. Network time reflects internet latency to OpenAI servers and is not a property of the model.

Method	Encode (ms)	Search (ms)	Network (ms)	Total (ms)	Self-hostable
BM25	0.1	104.0	0.0	104.1	Yes
MiniLM	68.9	12.0	0.0	80.9	Yes
BGE	16.6	27.8	0.0	44.4	Yes
OpenAI	<i>server</i>	42.9	211.7	254.6	No

BGE is the fastest method overall. BGE achieves a total query latency of 44.4 ms — faster than BM25 (104.1 ms), MiniLM (80.9 ms), and OpenAI (254.6 ms). This result is initially counterintuitive: BGE is a 335-million parameter neural network, whereas BM25 is a simple keyword scoring function. The explanation lies in hardware utilisation. BGE encodes the query vector on the A100 GPU (16.6 ms), a highly parallelised operation that is extremely efficient for dense linear algebra. BM25, by contrast, computes token scores on CPU over an inverted index (104.0 ms), which is sequential and memory-bound. This finding has a direct implication for production deployment: a GPU-equipped system running BGE delivers both better retrieval quality and lower query latency than a CPU-based BM25 system.

MiniLM encodes slowly despite being a small model. MiniLM’s 68.9 ms encoding time is slower than BGE’s 16.6 ms, despite having 15 times fewer parameters. This is because MiniLM encodes on CPU in these experiments, while BGE encodes on GPU. Hardware assignment dominates over model size for single-query encoding latency.

4. Experiments

OpenAI’s latency is dominated by network time. Of OpenAI’s total 254.6 ms, only 42.9 ms is FAISS index search — comparable to BGE’s 27.8 ms search time. The remaining 211.7 ms is the HTTP round-trip to OpenAI’s servers. This latency is not a property of the embedding model; it reflects internet connectivity between the university cluster and OpenAI’s infrastructure, and would vary across deployment environments. For a production system at Istari, API-based encoding would introduce an irreducible network dependency and per-query cost, whereas BGE could be self-hosted with predictable latency.

Summary. Table 4.3 summarises the key trade-offs across all four methods.

Table 4.3.: Summary comparison of all baseline retrieval methods.

Method	NDCG@10	Prec@10	Recall@1k	Latency	Cost	GPU req.
BM25	0.166	0.156	0.682	104 ms	Free	No
MiniLM	0.192	0.186	0.653	81 ms	Free	No
BGE	0.227	0.216	0.796	44 ms	Free	Yes (enc.)
OpenAI	0.227	0.224	0.803	255 ms	\$0.02/M tok.	No

BGE emerges as the recommended baseline for subsequent experiments: it matches OpenAI’s retrieval quality, is the fastest method in practice, is free, and can be fully self-hosted without API dependency. The results motivate the next experimental phase, in which a pre-filtering stage and hybrid retrieval system are built on top of the BGE baseline to further improve precision at low cut-offs and overall system efficiency.

4.6. Evaluation Protocol Change for Follow-Up Experiments

All experiments described in the remainder of this chapter (Sections 4.7–4.15) use a **relaxed pseudo-relevance definition**: a company counts as relevant if it appears anywhere in production’s **top-1 000** results for a query, rather than the top-100 threshold used in Section 4.1.2 and Table 4.1. The relaxed threshold was adopted so that reranking, fusion, and oracle-ceiling experiments would have enough relevant companies per query to produce meaningful comparisons over large candidate pools. As a direct consequence, the absolute NDCG/Precision/Recall values reported from here on are on a different (generally much higher) numerical scale than Table 4.1, and **are not directly comparable to it**. All comparisons in the following sections should be read only relative to one another, within the same section.

4.7. Extended Baseline Comparison: Additional Embedding Models

Five further dense embedding models were evaluated on the same 98 716-company corpus, all-fields text representation, and 101-query evaluation set described in Section 4.1.1:

- **BGE-M3** (BAAI/bge-m3): a multi-lingual, multi-functionality embedding model, 1 024-dimensional embeddings.
- **GTE-large** (thenlper/gte-large): a general-purpose text embedding model, 1 024 dimensions.
- **E5-Mistral-7B-Instruct** (intfloat/e5-mistral-7b-instruct): an instruction-tuned 7-billion-parameter Mistral-based embedding model, 4 096 dimensions.
- **Linq-Embed-Mistral** (Linq-AI-Research/Linq-Embed-Mistral): a 7B-parameter Mistral-based embedding model fine-tuned for retrieval, 4 096 dimensions.
- **SFR-Embedding-Mistral** (Salesforce/SFR-Embedding-Mistral): Salesforce’s 7B-parameter Mistral-based retrieval embedding model, 4 096 dimensions.

Table 4.4.: Extended baseline results under the relaxed (top-1 000) pseudo-relevance protocol (Section 4.6). Bold indicates the best value in each column.

Method	NDCG@ <i>k</i>				Recall@ <i>k</i>			
	10	50	100	1000	10	50	100	1000
BGE-M3	0.935	0.921	0.911	0.732	0.009	0.046	0.091	0.704
GTE-large	0.946	0.937	0.929	0.790	0.010	0.047	0.093	0.768
E5-Mistral-7B	0.631	0.587	0.544	0.303	0.006	0.029	0.052	0.274
Linq-Mistral	0.936	0.925	0.918	0.772	0.009	0.046	0.091	0.749
SFR-Mistral	0.938	0.926	0.919	0.770	0.009	0.046	0.092	0.746

Findings. GTE-large is the strongest of the five additional models (NDCG@10 = 0.946, Recall@1000 = 0.790), ahead of SFR-Mistral (0.938 / 0.770), Linq-Mistral (0.936 / 0.772), and BGE-M3 (0.935 / 0.732). These four sit close together and clearly ahead of E5-Mistral-7B-Instruct, which substantially underperforms (NDCG@10 = 0.631, Recall@1000 = 0.303) despite sharing the same base architecture and parameter count as Linq-Mistral and SFR-Mistral – indicating that fine-tuning data and strategy, not model size, are the dominant factor for this retrieval task. Notably, the two much smaller 1 024-dimensional models (BGE-M3, GTE-large) are competitive with or better than the 7B-parameter models, at a fraction of the encoding cost.

4. Experiments

Encoding cost. The three 7B-parameter models required substantially more compute to encode the corpus: E5-Mistral took 2011 seconds (33.5 min), Linq-Mistral 2039 seconds (34.0 min), and SFR-Mistral 2032 seconds (33.9 min), all on an A100 GPU, and all requiring checkpoint/resume logic to fit within the cluster’s 30-minute job wall-time limit. This is roughly five times the encoding time of the original 335M-parameter BGE-large baseline (7.0 minutes, Section 4.3.2), for retrieval quality that is not better and, in E5-Mistral’s case, substantially worse.

Incomplete and abandoned models. Two further large embedding models were attempted but did not produce final results. **BGE-Multilingual-Gemma2** (BAAI/bge-multilingual-gemma2, 9B parameters) encoding is still in progress at the time of writing: model loading alone takes 15–27 minutes on this cluster’s shared storage (highly variable due to filesystem I/O contention), leaving little of the 30-minute job window for actual encoding, so the job must be resubmitted repeatedly to make incremental progress via checkpointing. **NV-Embed-v2** (nvidia/NV-Embed-v2) was abandoned after a persistent `AttributeError` caused by an incompatibility between the model’s custom code and the installed `transformers` library version, which could not be resolved without a library downgrade that risked breaking the other baselines’ environment.

4.8. BGE Instruction-Prefix Correction

The original BGE-large baseline (Section 4.3.2) encoded queries without an instruction prefix. BGE-large’s model card specifies that, for retrieval tasks, each *query* should be prefixed with the instruction “*Represent this sentence for searching relevant passages:* ” before encoding; the passage (company) side requires no prefix, so the cached company embeddings could be reused without re-encoding the corpus. This fix was retested on both the all-fields and summary-only variants of the BGE baseline.

Table 4.5.: Effect of the query instruction prefix on BGE-large, measured under the relaxed top-1 000 protocol (Section 4.6).

Variant	NDCG@10	Recall@1000	Δ NDCG@10	Δ Recall@1000
All-fields, no prefix	0.942	0.726	–	–
All-fields, with prefix	0.937	0.727	–0.005	+0.001
Summary-only, no prefix	0.937	0.700	–	–
Summary-only, with prefix	0.924	0.707	–0.014	+0.007

Finding. Contrary to expectation, applying the officially documented query prefix produced only a marginal and mixed effect on this dataset: Recall@1000 improved negligibly (+0.001 for all-fields, +0.007 for summary-only), while NDCG@10 slightly *decreased* in both variants. Both effects are small enough to plausibly be within query-level noise. This suggests that, for this company-retrieval task, BGE-large’s query prefix convention

4. Experiments

– tuned primarily for open-domain passage retrieval benchmarks – does not transfer into a measurable improvement, and the model’s underlying limitation observed in Section 4.3.2 (underperforming MiniLM on Recall@1000) is not explained by the missing prefix.

4.9. Query Expansion: HyDE Pipeline

HyDE (Hypothetical Document Embeddings) expands each query by asking a local LLM (llama3.1:8b via Ollama) to generate $N = 10$ stochastic (temperature = 0.8) hypothetical company descriptions that would ideally answer the query. The embeddings of these 10 descriptions are averaged into a single query vector, which is then searched against the MiniLM company index. A second stage reranks the top-50 HyDE results with a cross-encoder (BAAI/bge-reranker-v2-m3).

Table 4.6.: HyDE pipeline results (relaxed top-1 000 protocol) compared against the plain MiniLM baseline re-evaluated under the same protocol.

Method	NDCG@10	Recall@1000	Latency (ms)
MiniLM (no expansion)	0.975	0.741	16.9
Stage 1: HyDE + MiniLM	0.829	0.596	9 246.9
Stage 2: HyDE + MiniLM + Reranker	0.954	0.596	9 488.3

Finding. Query expansion via HyDE *hurts* retrieval quality relative to directly encoding the original query: Stage 1 NDCG@10 drops from 0.975 to 0.829, and Recall@1000 drops from 0.741 to 0.596. Adding a cross-encoder reranker (Stage 2) recovers most of the NDCG@10 loss ($0.829 \rightarrow 0.954$) by re-ordering the top-50 candidates, but cannot recover the recall loss, since reranking only reorders an already -fixed candidate set. Latency is dominated overwhelmingly by the LLM call itself (9.2 seconds), three orders of magnitude slower than directly encoding the query with MiniLM (16.9 ms). For this dataset, HyDE is not a worthwhile query-expansion strategy: it is far slower and retrieves a worse candidate set than simply embedding the query directly.

4.10. Hybrid Retrieval: Reciprocal Rank Fusion

As an alternative to query expansion, BM25 and MiniLM rankings are fused directly using Reciprocal Rank Fusion (RRF), $\text{RRF}(d) = \sum_{\text{method}} \frac{1}{k + \text{rank}(d)}$ with $k = 60$ (Cormack et al. 2009). A second stage reranks the top-50 fused results with the same cross-encoder used for HyDE.

Finding. Unlike HyDE, RRF fusion of BM25 and MiniLM adds negligible latency (the fusion step itself takes 2.2 ms) and, after reranking, matches the plain MiniLM baseline’s

4. Experiments

Table 4.7.: Hybrid RRF results (relaxed top-1 000 protocol).

Method	NDCG@10	Recall@1000	Latency (ms)
MiniLM (no fusion)	0.975	0.741	16.9
Stage 1: RRF (BM25 + MiniLM)	0.955	0.724	104.9
Stage 2: RRF + Reranker	0.975	0.724	352.0

NDCG@10 (0.975) while Recall@1000 remains slightly below MiniLM alone (0.724 vs 0.741) – BM25’s keyword matches do not add companies beyond what MiniLM already retrieves for this query set, so fusion mostly re-confirms rather than extends MiniLM’s candidate pool. RRF is a much cheaper and more effective way to incorporate a second retrieval signal than HyDE-style query expansion.

4.11. LLM-as-Judge Relevance Validation

Because the pseudo-relevance labels (Section 4.1.2) assume production’s own rankings are correct, an independent check was run using an LLM to directly judge relevance. For 20 held-out queries (from a pool of 101, split 81/20 train/held-out), the top-20 production results per query were judged individually on a 0/1/2 relevance scale by an LLM given the query and the company summary, yielding 400 graded judgements in total.

Table 4.8.: NDCG under pseudo-relevance labels vs. LLM-judged relevance labels, on the 20 held-out queries.

k	Pseudo-relevance NDCG	LLM-judged NDCG	Δ
10	0.964	0.963	−0.001
50	0.947	0.982	+0.035
100	0.939	0.982	+0.042
500	0.868	0.982	+0.113
1000	0.771	0.982	+0.210

Finding. At $k = 10$, pseudo-relevance and LLM-judged NDCG agree closely (0.964 vs 0.963), supporting the use of production’s own top results as a reasonable relevance proxy at shallow depths. The two diverge sharply as k grows: LLM-judged NDCG stays essentially flat at 0.982 out to $k = 1000$, while pseudo-relevance NDCG falls to 0.771. This is consistent with the pseudo-relevance labels being a hard top-100 cutoff (Section 4.1.2): once retrieval looks past that cutoff, pseudo-relevance necessarily penalises the system even for companies an LLM judge considers genuinely relevant. This supports the interpretation, already flagged in Section 4.1.2, that pseudo-relevance NDCG at large k understates true system quality rather than reflecting a real quality gap.

4.12. Production API Evaluation and MiniLM Rerank

Two rounds of evaluation compared the local pipeline against Istari’s live production search API, using a separate ground truth (the production system’s own graded semantic-search results at a similarity threshold of 0.75) rather than the pseudo-relevance labels used elsewhere in this chapter.

v1: keyword API. The first round compared Istari’s v1 (BM25-based) production API against local MiniLM, both queried against the same 101 queries. Istari v1 BM25 NDCG@10 was 0.101 and Recall@1000 was 0.109 (network latency 2828 ms/query), against local MiniLM’s NDCG@10 of 0.935 and Recall@1000 of 0.766 (local GPU latency 26.6 ms/query) – confirming that the production keyword API is a much weaker retrieval signal than even the simplest local dense baseline.

v2: semantic API. A second round added Istari’s newer v2 semantic search API (live, paginated over the full 20-million-company GOI, quota-aware). Istari v2 NDCG@10 was 0.985 and Recall@1000 (at the achievable depth of 816 results) was 0.650, clearly ahead of local MiniLM (NDCG@10 0.935, Recall@816 0.709) at shallow cutoffs, though local MiniLM’s Recall catches up and slightly exceeds v2’s at $k = 816$. Network latency for v2 (1966 ms/query for a full fetch) remains far higher than local MiniLM (26.4 ms/query).

Reranking v2’s own candidates with MiniLM. A follow-up experiment took v2’s own retrieved candidate pool and re-ordered it locally using MiniLM, to test whether local reranking could improve on v2’s native ranking without changing which companies are retrieved. This made results *worse*: NDCG@10 fell from 0.985 (v2 native ranking) to 0.855 (MiniLM-reranked). Istari’s v2 semantic ranking is not simply a MiniLM-style dense embedding search under the hood – it outperforms MiniLM’s own ranking of the same candidates.

4.13. ANN Index Scaling to 2 Million Vectors

To test whether the exact search used throughout this chapter (`IndexFlatL2/IndexFlatIP`) remains practical at production scale, the 98 716 real MiniLM company embeddings were padded with 1 901 284 synthetic filler vectors (real embeddings perturbed with Gaussian noise at 15% of the average vector norm), giving a 2-million-vector corpus. Exact search (`FlatL2`) was compared against approximate indexes: IVF (`nlist=4096`, with `nprobe` $\in \{1, 8, 32, 128, 4096\}$) and HNSW ($M = 32$, with `efSearch` $\in \{16, 64, 256\}$).

Finding. Both IVF (`nprobe=8`) and HNSW (`efSearch=256`) recover essentially all of exact search’s NDCG@10, and IVF slightly exceeds exact search’s Recall@1000 (0.743 vs 0.741, within noise), while running 6.5–6.6 $\times$ faster at 2-million-vector scale, at the cost of modest additional index memory (0.7% and 17.7% larger, respectively). This confirms that approximate search is a safe and substantial efficiency gain for scaling the

4. Experiments

Table 4.9.: Exact vs. approximate search at 2M-vector scale (relaxed top-1 000 protocol).
Only representative configurations shown.

Index	NDCG@10	Recall@1000	ms/query	Speedup	Index size
Exact (FlatL2)	0.975	0.741	175.5	1.0×	3 072 MB
IVF (nprobe=8)	0.974	0.743	26.4	6.6×	3 094 MB
HNSW (efSearch=256)	0.975	0.727	26.9	6.5×	3 616 MB

retrieval system toward the full 20-million-company GOI, without a meaningful quality trade-off at this scale.

4.14. Recall@1000 Error Analysis

To understand why exact MiniLM search caps at Recall@1000 = 0.741 (Section 4.13), every production-relevant company that MiniLM failed to retrieve within the top-1 000 was extracted and searched for at much greater depth ($k = 10\,000$), yielding 26 185 missed (query, company) pairs across all 101 queries.

Table 4.10.: Classification of the 26 185 Recall@1000 misses by how far beyond rank 1 000 the true company was actually ranked.

Bucket	Count	Share (%)
Near miss (rank 1 000–2 000)	16 157	61.7
Mid miss (rank 2 000–5 000)	8 061	30.8
Far miss (rank 5 000–10 000)	1 375	5.3
Not found (beyond rank 10 000)	592	2.3

Finding. The large majority of misses (61.7%) are near misses, just outside the top-1 000 cutoff, suggesting most of the recall gap is a matter of ranking granularity rather than a hard semantic failure. Only 2.3% of misses are not found at all within $k = 10\,000$, indicating genuine embedding failures are rare. Separately, 3.8% of missed companies (996 of 26 185) were flagged as boilerplate content, and one extreme case of description duplication was found: the placeholder text “Bluehost.com” appears as the company description for 709 distinct domains, meaning these domains are indistinguishable to any text-embedding model and can never be individually retrieved correctly. This is a concrete data-quality artifact rather than a retrieval-model limitation, and sets a hard ceiling on achievable recall for any embedding-based method operating on the `summary` field alone.

4.15. Oracle Union Recall Ceiling

Before investing in a trained reranker or a fine-tuning pipeline, it is useful to know the best-case recall achievable by simply combining the candidate pools already produced by every method run so far — at zero additional encoding cost, since every channel’s top-1 000 results per query are already cached on disk. This experiment computes, for every combination of the six available channels (MiniLM, BM25, BGE-all-fields, BGE-summary, OpenAI-large, Nomic), the recall of the *union* of their top-1 000 candidate sets against the top-1 000 ground truth.

Findings. The best single channel (MiniLM alone) reaches $\text{Recall@1000} = 0.741$. The full union of all six channels reaches 0.936 (average combined pool size 1 949 candidates/query), a gain of +0.196 over MiniLM alone. The best-performing *pair* is MiniLM + OpenAI-large, at 0.856 (average pool size 1 300). A pool-size sweep on this best pair shows the ceiling is highly sensitive to how many candidates are taken per channel: at 100 candidates/channel the union only reaches 0.150 recall, rising to 0.581 at 500, and 0.856 at the full 1 000.

Interpretation. Because the six-channel union clears 0.936 – well above any individual channel – the different retrieval methods are largely making *complementary*, not correlated, errors: a reranker operating over a modestly sized union pool (rather than a single channel) is likely sufficient to approach this ceiling, and a full fine-tuning pipeline is not obviously necessary to close the remaining gap. The residual 0.064 recall lost even under the full six-channel union is consistent with the hard data-quality ceiling identified in Section 4.14 (duplicated/boilerplate company descriptions that no embedding-based channel can disambiguate).

5. Introduction

The structure of this example thesis is *non-binding*, but is highly topic-dependent and should be discussed with your advisor. You probably do not want to go below the level of subsections, though.

To cite, do it in one of the following ways:

- A single paper in text: As shown by Doe (2024), X holds.
- Same with additional information: As shown by Doe (2024, p. 20), X holds.
- A single paper in parenthesis: X holds (Doe 2024).
- Multiple papers in parenthesis: X holds (Brown 2022; Smith and Johnson 2023; Lee and Green 2023; Doe 2024).
- Use footnotes to reference a website, such as the DWS thesis guidelines.¹ Note that footnote marks are set after punctuation in English text. Only reference a website when needed—e.g., the website of a library or a blog post—and always cite corresponding scientific publications (if any) in addition.

Format the bibliography consistently, e.g., as in the the example bibliography in this template. Only cite archival versions of papers (such as those published on arXiv) when there are no corresponding official publications (such as in conference proceedings); if there are such publications, then cite those instead.

Some examples of references are:

1. Table 9.1 is an example table.
2. Section 6.1 is a later section of this thesis.
3. Algorithm 8.1 is an example algorithm.
4. And finally, Equation (5.1) is an example equation.

Here is the equation referred to above:

$$\mathcal{S} = \{ a \cdot \text{relu}(\mathbf{W}^\top \mathbf{x}) : a \in \mathbb{R} \} \quad (5.1)$$

Important: Fill out and sign the last page when you submit/deliver the final version of your thesis. Otherwise, your work cannot be accepted for legal reasons.

The purpose of the introduction as summarized by Zobel (2004):

¹<https://www.uni-mannheim.de/dws/teaching/thesis-guidelines/>

5. Introduction

An introduction can be regarded as an expanded version of the abstract. It should describe the paper's topic, the problem being studied, references to key papers, the approach to the solution, the scope and limitations of the solution, and the outcomes. There needs to be enough detail to allow readers to decide whether or not they need to read further. It should include motivation: the introduction should explain why the problem is interesting, what the relevant scientific issues are, and why the solution is a good one.

That is, the introduction should show that the paper is worth reading and it should allow the reader to understand your perspective, so that the reader and you can proceed on a basis of common understanding.

Many introductions follow a five-element organization:

1. A general statement introducing the broad research area of the particular topic being investigated.
2. An explanation of the specific problem (difficulty, obstacle, challenge) to be solved.
3. A brief review of existing or standard solutions to this problem and their limitations.
4. An outline of the proposed new solution.
5. A summary of how the solution was evaluated and what the outcomes of the evaluation were.

An interesting exercise is to read other papers, analyze their introductions to see if they have this form, and then decide whether they are effective. The introduction can discuss the importance or ramifications of the conclusions but should omit supporting evidence, which the interested reader can find in the body of the paper. Relevant literature can be cited in the introduction, but unnecessary jargon, complex mathematics, and in-depth discussion of the literature belong elsewhere.

A paper isn't a story in which results are kept secret until a surprise ending. The introduction should clearly tell the reader what in the paper is new and what the outcomes are. There may still be a little suspense: revealing what the results are does not necessarily reveal how they were achieved. If, however, the existence of results is concealed until later on, the reader might assume there are no results and discard the paper as worthless. (Zobel 2004)

6. Literature Review

Zobel (2004) writes:

Few results or experiments are entirely new. Most often they are extensions of or corrections to previous research—that is, most results are an incremental addition to existing knowledge. A literature review, or survey, is used to compare the new results to similar previously published results, to describe existing knowledge, and to explain how it is extended by the new results. A survey can also help a reader who is not expert in the field to understand the paper and may point to standard references such as texts or survey articles.

In an ideal paper, the literature review is as interesting and thorough as the description of the paper's contribution. There is great value for the reader in a precise analysis of previous work that explains, for example, how existing methods differ from one another and what their respective strengths and weaknesses are. Such a review also creates a specific expectation of what the contribution of the paper should be—it shapes what the readers expect of your work, and thus shapes how they will respond to your ideas.

The literature review can be early in a paper, to describe the context of the work, and might in that case be part of the introduction; or the literature review can follow or be part of the main body, at which point a detailed comparison between the old and the new can be made. If the literature review is late in a paper, it is easier to present the surveyed results in a consistent terminology, even when the cited papers have differing nomenclature and notation. In many papers the literature review material is not gathered into a single section, but is discussed where it is used—background material in the introduction, analysis of other researchers' work as new results are introduced, and so on. This approach can help you to write the paper as a flowing narrative.

An issue that is difficult in some research is the relationship between new scientific results and proprietary commercial technology. It often is the case that scientists investigate problems that appear to be solved or addressed in commercial products. For example, there is ongoing academic research into methods for information retrieval despite the success of the search engines deployed on the web. From the perspective of high research principle, the existence of a commercial product is irrelevant: the ideas are not in the public domain, it is not known how the problems were solved in the product, and the researcher's contribution is valid. However, it may well be reckless

6. *Literature Review*

to ignore the product; it should be cited and discussed, while noting, for example, that the methods and effectiveness of the commercial solution are unknown. (Zobel 2004)

An example structure for the literature review is given below; as before, this structure is non-binding and should be changed as appropriate.

6.1. Preliminaries

6.2. Related Work on A

6.3. Related Work on B

6.4. Related Work on C

6.5. Summary

7. Body Chapter 1

Zobel (2004) writes:

The body of a paper should present the results. The presentation should provide necessary background and terminology, explain the chain of reasoning that leads to the conclusions, provide the details of central proofs, summarize any experimental outcomes, and state in detail the conclusions outlined in the introduction. Descriptions of experiments should permit reproduction and verification, as discussed in Chapter 11. There should also be careful definitions of the hypothesis and major concepts, even those described informally in the introduction. The structure should be evident in the section headings. Since the body can be long, narrative flow and a clear logical structure are essential.

The body should be reasonably independent of other papers. If, to understand your paper, the reader must find specialized literature such as your earlier papers or an obscure paper by your advisor, then its audience will be limited.

In some disciplines, research papers have highly standardized structures. Editors may require, for example, that you use only the four headings Introduction-Methods-Results-Discussion. This convention has not taken hold in computer science, and in some cases such a structure impedes a clear explanation of the work. For example, use of fixed headings may prohibit development of a complex explanation in stages. In work combining two query resolution techniques, we had to determine how they would interact, based on a fresh evaluation of how they behaved independently. The final structure was, in effect, Introduction-Background-Methods-Results-Discussion-Methods-ResultsDiscussion.

Even if the standardized section names are not used, the body needs these elements, if not necessarily under their standard headings. Components of the body might include, among other things, background, previous work, proposals, experimental design, analysis, results, and discussion. Specific research projects suggest specific headings. For the "compression for fast external sorting" project sketched earlier, the complete set of section headings might be:

1. Introduction
2. External sorting

7. *Body Chapter 1*

3. Compression techniques for database systems
4. Sorting with compression
5. Experimental setup
6. Results and discussion
7. Conclusions

The wording of these headings does not follow the standard form, but the intent of the wording is the same. Sections 2 and 3 are the background; Section 4 contains novel algorithms, and Sections 4 and 5 together are the methods.

The background material can be entirely separate from the discussion of previous work on the same problem. The former is the knowledge the reader needs to understand your contribution. The latter is, often, alternative solutions that are superseded by your work. Together, the discussion of background and previous work also introduce the state of the art and its failings, the importance and circumstances of the research question, and benchmarks or baselines that the new work should be compared to.

A body that consists of descriptions of algorithms followed by a dump of experimental results is not sound science. In such a paper, the context of prior work is not explained, as readers are left to draw their own inferences about what the results mean.

In a thesis, each chapter has structure, including an introduction and a summary or conclusions. This structure varies with the chapter's purpose. A background chapter may gather a variety of topics necessary to understanding of the contribution of the thesis, for example, whereas a chapter on a new algorithm may have a simple linear organization in which the parts of the algorithm are presented in turn. However, the introduction and summary should help to link the thesis together-how the chapter builds on previous chapters and how subsequent chapters make use of it. (Zobel 2004)

8. Body Chapter 2

Our key result is given as Algorithm 8.1. It's complete non-sense, of course! Note that algorithms, figures, and tables should generally be placed at the top of the page or on an individual page, but not within the main text.

The following highly informative text is here so that the correct placement of Algorithm 8.1 is visible.

Algorithm 8.1 Proof that X Holds

Require: A hypothesis H that X holds

Ensure: A proof that X holds

```

1:  $P \leftarrow \emptyset$                                 // Initialize proof as an empty set of statements
2:  $A \leftarrow$  Assumptions from hypothesis  $H$         // Extract assumptions from  $H$ 
3: for all  $a \in A$  do
4:   if  $a$  is a known axiom then
5:     Add  $a$  to  $P$ 
6:   else if  $a$  can be derived from known axioms then
7:     Derive  $a$  from axioms and add to  $P$ 
8:   else
9:     fail                                           // If any assumption cannot be derived, proof fails
10:  end if
11: end for
12: Use logical deductions on  $P$  to prove intermediate results
13: Combine intermediate results to prove  $X$ 
14: return  $P$                                          // Return the completed proof

```

9. Experimental Evaluation

9.1. Experimental Setup

9.2. Results 1

9.3. Results 2

9.4. Results 3

9.5. Discussion

Item	Quantity	Price
Apples	10	\$2.00
Oranges	5	\$3.00
Bananas	7	\$1.50

Table 9.1.: Sample table using booktabs

10. Conclusions

Zobel (2004) writes:

The closing section, or summary, is used to draw together the topics discussed in the paper. It should include a concise statement of the paper's important results and an explanation of their significance. This is an appropriate place to state (or restate) any limitations of the work: shortcomings in the experiments, problems that the theory does not address, and so on.

The conclusions are an appropriate place for a scientist to look beyond the current context to other problems that were not addressed, to questions that were not answered, to variations that could be explored. They may include speculation, such as discussion of possible consequences of the results.

A *conclusion* is that which concludes, or the end. *Conclusions* are the inferences drawn from a collection of information. Write "Conclusions", not "Conclusion". If you have no conclusions to draw, write "Summary". (Zobel 2004)

Bibliography

- Steven Bird, Ewan Klein, and Edward Loper. NLTK: the natural language toolkit, 2009.
- Emily Brown. A technical report on novel proofs for X. Technical Report TR-2022-09, Hypothetical Research Institute, Eureka, 2022.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- Jane Doe. *The Comprehensive Guide to Proving X*, volume 3. Fictional Academic Press, Springfield, 2nd edition, 2024. ISBN 123-4567890123.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- Michael Lee and Sarah Green. A comprehensive study on the validity of X. *Journal of Hypothetical Studies*, 15(4):123–145, July 2023. doi: 10.5678/jhs.2023.154.123.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- John Smith and Alice Johnson. Proof techniques for X. In *Proceedings of the International Symposium on Incorrect Proofs*, pages 45–58, Atlantis, 2023. Symposium Publications. doi: 10.1234/symp2023.45.

Bibliography

- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Ellen M Voorhees. The trec-8 question answering track report. In *Proceedings of TREC*, 1999.
- Justin Zobel. *Writing for Computer Science*. Springer London, 2nd edition, 2004. doi: <https://doi.org/10.1007/978-0-85729-422-7>.
- Justin Zobel. *Writing for Computer Science*. Springer London, 3rd edition, 2014. doi: <https://doi.org/10.1007/978-1-4471-6639-9>.

A. Additional Experimental Results

Zobel (2004) writes:

Some papers have appendices giving detail of proofs or experimental results, and, where appropriate, material such as listings of computer programs. The purpose of an appendix is to hold bulky material that would otherwise interfere with the narrative flow of the paper, or material that even interested readers do not need to refer to. Appendices are rarely necessary. (Zobel 2004)

In the context of a BSc or MSc thesis, the last sentence often does not hold.

B. Proof Details

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools			
Tool	Purpose	Where?	Useful?
ChatGPT	Rephrasing	Throughout	+
DeepL	Translation	Throughout	+
ResearchGPT	Summarization of related work	Sec. 6.2	-
Dall-E	Image generation	Figs. 2, 3	++
GPT-4	Code generation	functions.py	+
ChatGPT	Related work hallucination	Most of bibliography	++

Unterschrift
Mannheim, den July 7, 2026